DSA ASSIGNMENT REPORT

Introduction

Algorithms are an important part of computer science because they help us solve problems in a systematic way. In this assignment, we study different recursive algorithms and analyse their efficiency.

Recursion is a technique where a function calls itself to solve smaller parts of a problem. Some problems like factorial, Fibonacci series, and Tower of Hanoi are easier to solve using recursion.

In this assignment, I implemented a small toolkit called **Algorithmic Efficiency and Recursion Toolkit (AERT)** using Python. This toolkit demonstrates important algorithmic concepts such as recursion, memoization, stack operations, and binary search.

Explanations of all tasks/sub tasks assigned

Task 1 – Stack Abstract Data Type (ADT)

Q: What is an ADT?

An Abstract Data Type (ADT) is a concept that defines how a data structure behaves without explaining how it is implemented internally.

In simple words, an ADT only tells:

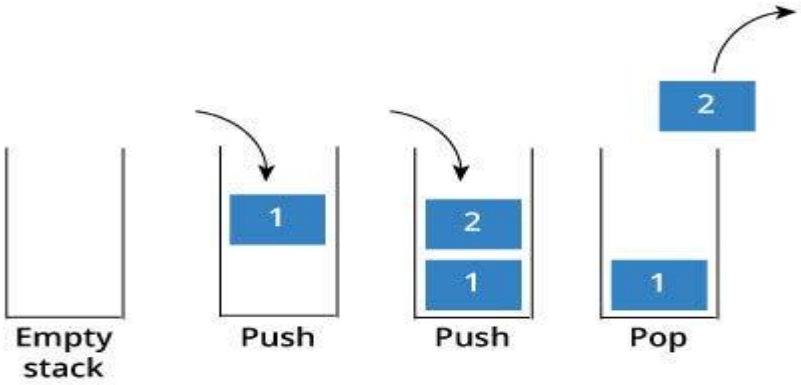
- what operations can be performed
- what those operations do

but it does not tell how they are implemented in code.

Q: Why Stack Operations are O(1) ?

Most stack operations like push, pop, and peek take constant time O(1).

This happens because these operations only affect the **top element** of the stack.



Stack.append(x) & Stack.pop() these operations do not require searching through the entire stack.

So no matter if the stack has 5 elements or 5000 elements, the time required is almost the same.

That is why stack operations usually have O(1) time complexity.

<u>Task 2 - Factorial and Fibonacci</u>

Q: Time and Space Complexity of Factorial ?

Each recursive call reduces the value by 1.So total number of calls =**n**

Therefore: Time Complexity =O(n)

Best Case: Ω(n)

Worst: Case O(n)

Theta: Θ(n)

<u>Space Complexity:</u> O(n) ,as recursion uses the **call stack**, every recursive call stays in memory. So recursion depth = **n.**

Q: Time and Space Complexity of Fibonacci ?

Naive Fibonacci

The naive recursive Fibonacci function repeatedly calculates the same values. This causes a large number of recursive calls.

Time Complexity: O(2^n)

Best Case: Ω(1)

Theta: Θ(2^n)

Space Complexity: O(n) as recursion depth = n

Memoized Fibonacci

Memoization stores previously calculated values so that they do not need to be calculated again. When the function needs a value again, it directly uses the stored value.

This greatly reduces the number of recursive calls.

Time Complexity: O(n)

Best Case: Ω(n)

Theta: Θ(n)

Space Complexity: O(n) Memoization table stores n values.

Q: Why Naive Fibonacci Becomes Slow for Large n ?

Naive Fibonacci becomes slow because it recomputes the same values many times.

For example: To compute fib(6)

fib(5) and fib(4) but inside fib(5), fib(4) is again calculated.

This repeated computation causes the number of recursive calls to grow exponentially. As n increases, the program becomes extremely slow.

Memoization solves this problem by saving already computed results.

Task 3 – Tower of Hanoi

Q: Why the Number of Moves is $2^n - 1$?

The Tower of Hanoi problem follows a recursive pattern.

To move n disks, we must:

1. Move n−1 disks to helper rod
2. Move the largest disk to destination
3. Move n−1 disks from helper to destination
4. So the recurrence relation becomes: T(n) = 2T(n-1) + 1

   Solving this gives: T(n) = 2^n – 1

- Each disk causes two recursive calls.
  So the number of operations grows exponentially.
- Time Complexity: O(2^n)
  Recursion depth = number of disks
- Space complexity: O(n)

Task 4 - Recursive Binary Search

Q: Define the recurrence relation and complexities in all the scenarios?

Binary Search works only on a sorted array. It divides the array into two halves and searches in one half.

Recurrence relation: T(n) = T(n/2) + O(1)

This means the problem size becomes half in every step.

Solving this recurrence gives: O(log n)

- **<u>Best Case</u>**
  Best case occurs when the element is **exactly the middle element**.
  Example: arr = [1,3,5,7,9] , search = 5
  The element is found in the first step.
  Best Case Complexity: Ω(1)
- **<u>Average Case</u>**
  In most cases, the element is found after several divisions.
  Average Complexity: O(log n)
- **<u>Worst Case</u>**
  Worst case happens when the element is:
  - not present in the list
  - or found at the last level of recursion

  Example: arr = [1,3,5,7,9], search = 8
  The algorithm will keep dividing until the list becomes empty.
  Worst Case Complexity: O(log n)

<u>Task 6 - Paradigm Reflection</u>

Q1: Why Binary Search uses Divide and Conquer approach?

Binary Search follows the Divide and Conquer paradigm.

It divides the problem into smaller parts.

<u>Steps</u>:

1. Divide the array into two halves
2. Conquer by searching in the correct half
3. Continue until the element is found

Example: [1,3,5,7,9,11,13]

- Searching **9**

❖ First divide → middle = 7
Then search right half → [9,11,13]

The problem size becomes smaller each step.

Q2: Why Memoized Fibonacci uses Dynamic Programming?

Dynamic Programming solves problems by storing solutions of smaller subproblems. Memoized Fibonacci stores previous Fibonacci values in memory.

Example: fib(5), needs fib(4) and fib(3)

But once fib(3) is calculated, it is saved.

So the program does not compute it again. This avoids repeated work and improves efficiency.

Q3: Why Tower of Hanoi Uses Recursion?

Tower of Hanoi works by solving a smaller version of the same problem.

Steps:

1. Move n−1 disks
2. Move largest disk
3. Move n−1 disks again

The base case is: n = 1

When there is only one disk, it can be moved directly. This pattern makes recursion suitable for this problem.

Real-Life Examples:

Divide and Conquer

- Searching for a word in a dictionary.
- Instead of checking every page, we open the **middle page** and decide whether to search the first half or the second half.
- This is similar to **binary search**.

Dynamic Programming

- Google Maps finding the **shortest route**.
- The system stores previously calculated paths and reuses them to avoid recomputation.

Greedy Algorithm

- ATM cash withdrawal. The ATM gives the largest denomination first.
- Example: For ₹270
  200 + 50 + 20
- It always chooses the best immediate option.

OUTPUT IMAGES

```
====== AERT TOOLKIT MENU ======
1. Calculate Factorial
2. Fibonacci (Naive)
3. Fibonacci (Memoization)
4. Tower of Hanoi
5. Recursive Binary Search
6. Exit
Enter your choice: 5
Enter sorted numbers separated by space: 2
Enter element to search: 4
Element not found

====== AERT TOOLKIT MENU ======
1. Calculate Factorial
2. Fibonacci (Naive)
3. Fibonacci (Memoization)
4. Tower of Hanoi
5. Recursive Binary Search
6. Exit
Enter your choice: 1
Enter a number: 10
Factorial = 3628800
```

```
PS C:\Users\s1420\Documents\VS code programs (DSA)> & C:\Users\s1420\AppData\Local\Microsoft\WindowsApps\python3.13.exe "c:/Users/s1420/Document
s/VS code programs (DSA)/Python_Assigments_Sem2/Assignment-1"

====== AERT TOOLKIT MENU ======
1. Calculate Factorial
2. Fibonacci (Naive)
3. Fibonacci (Memoization)
4. Tower of Hanoi
5. Recursive Binary Search
6. Exit
Enter your choice: 2
Enter n: 12
Fibonacci = 144
Number of recursive calls = 465

====== AERT TOOLKIT MENU ======
1. Calculate Factorial
2. Fibonacci (Naive)
3. Fibonacci (Memoization)
4. Tower of Hanoi
5. Recursive Binary Search
6. Exit
Enter your choice: 3
Enter n: 12
Fibonacci = 144
Number of recursive calls = 23
```

```
PS C:\Users\s1420\Documents\VS code programs (DSA)> & C:\Users\s1420\AppData\Local\Microsoft\WindowsApps\python3.13.ex
s/VS code programs (DSA)/Python_Assigments_Sem2/Assignment-1"

====== AERT TOOLKIT MENU ======
1. Calculate Factorial
2. Fibonacci (Naive)
3. Fibonacci (Memoization)
4. Tower of Hanoi
5. Recursive Binary Search
6. Exit
Enter your choice: 4
Enter number of disks: 3
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Total moves stored in stack: 7
```

```
PS C:\Users\s1420\Documents\VS code programs (DSA)> & C:\Users\s1420\AppData\Local\Microsoft\WindowsApps\python3.13.exe "c:/Users/s1420/Document
s/VS code programs (DSA)/Python_Assigments_Sem2/Assignment-1"

====== AERT TOOLKIT MENU ======
1. Calculate Factorial
2. Fibonacci (Naive)
3. Fibonacci (Memoization)
4. Tower of Hanoi
5. Recursive Binary Search
6. Exit
Enter your choice: 4
Enter number of disks: 3
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Total moves stored in stack: 7

====== AERT TOOLKIT MENU ======
1. Calculate Factorial
2. Fibonacci (Naive)
3. Fibonacci (Memoization)
4. Tower of Hanoi
5. Recursive Binary Search
6. Exit
Enter your choice: 4
Enter number of disks: 2
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C
Total moves stored in stack: 3
```

Work submitted By : Priya 2501720448